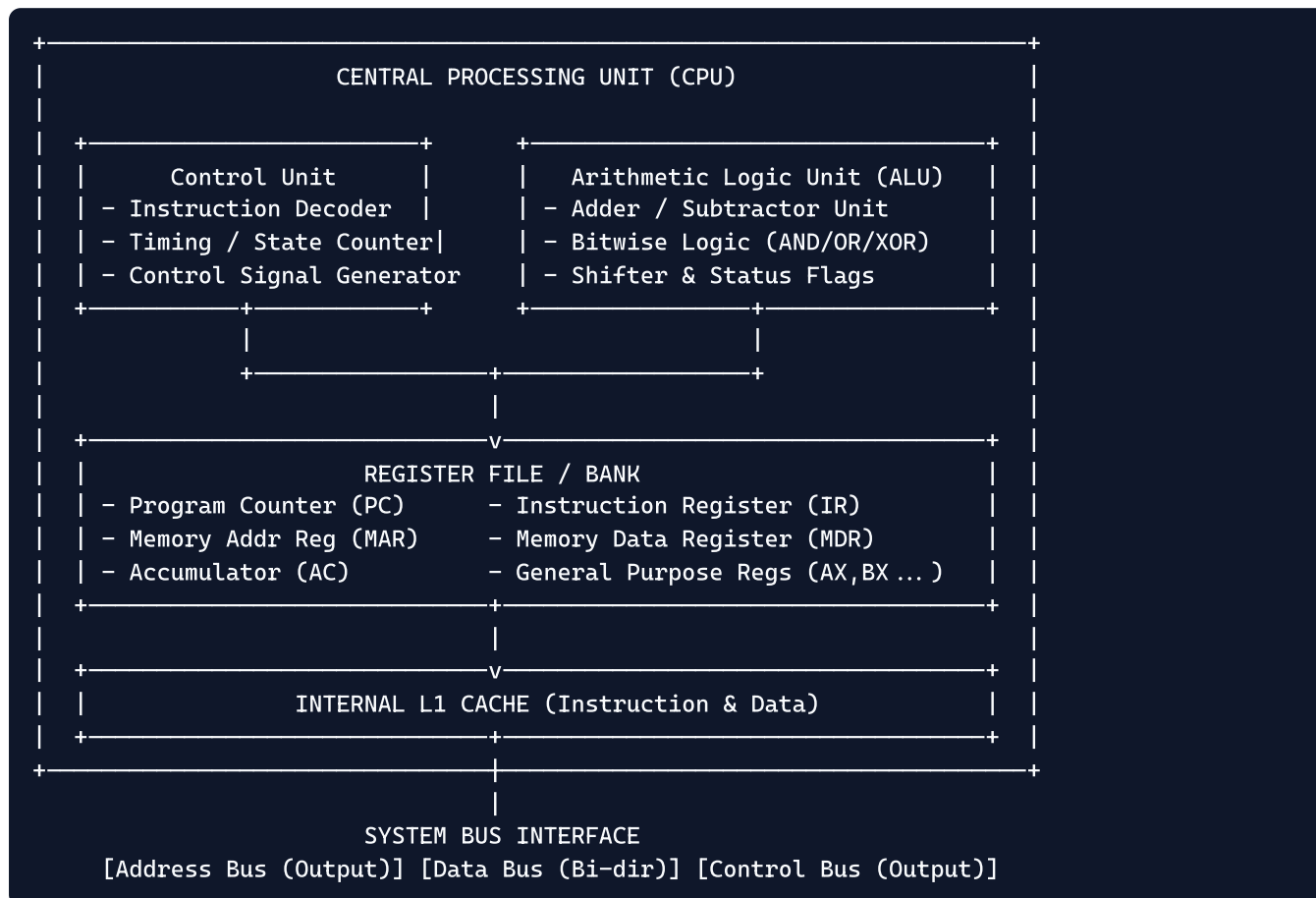


MPCA Module 2 — 10 Mark Questions & Answers

Q1. Explain CPU architecture in detail with a block diagram — covering ALU, Control Unit, Registers, Cache Memory, and Bus System with their functions.

1. Comprehensive CPU Block Diagram



2. Functional Unit Details

1. Control Unit (CU):

- * **Functions:** Directs the operational flow of the system.
- * Fetches instruction bytes, decodes opcodes, and emits synchronous micro-operation control pulses to registers, ALU, and system buses.

2. Arithmetic Logic Unit (ALU):

- * **Functions:** Executes high-speed mathematical operations.
- * Performs integer arithmetic (ADD, SUB, MUL, DIV) and logic evaluations (AND, OR, XOR, NOT, Bit-shifts). Updates status flags (ZF, CF, SF, OF).

3. Register Array / File:

- * **Program Counter (PC):** Holds 16/32-bit address of next instruction.
- * **Instruction Register (IR):** Holds active opcode being decoded.
- * **MAR & MDR:** Interface memory address lines and data lines.
- * **General Registers:** Provide ultra-fast temporary storage for ALU operands.

4. Cache Memory (L1 / L2):

- * High-speed SRAM placed inside CPU chip to store frequently accessed instructions and data, reducing slow DRAM main memory fetch delays.

5. System Bus Architecture:

- * **Address Bus (Unidirectional):** Carries physical memory/IO addresses from CPU.
- * **Data Bus (Bidirectional):** Transfers data bytes between CPU, memory, and devices.
- * **Control Bus (Multidirectional):** Transmits command/status signals (Read, Write, Interrupt Acknowledge, Clock).

Q2. Explain all four types of instruction formats (three-address, two-address, one-address, zero-address) in detail with structure, examples, advantages, and disadvantages.

1. Format Structures

```
3-Address: [ Opcode | Dest_Addr | Src1_Addr | Src2_Addr ]
2-Address: [ Opcode | Dest_Addr | Src_Addr ]
1-Address: [ Opcode | Src_Addr ]           (Dest = Implicit Accumulator)
0-Address: [ Opcode ]                     (Operands = Implicit Stack Top)
```

2. Program Execution Comparison for $X = (A + B) * (C + D)$

A. Three-Address Instruction Format

```
ADD R1, A, B    ; R1 = Mem[A] + Mem[B]
ADD R2, C, D    ; R2 = Mem[C] + Mem[D]
MUL X, R1, R2   ; Mem[X] = R1 * R2
```

- **Pros:** Shortest program length (fewer instructions); clear operand isolation.
- **Cons:** Instruction word requires many bits; complex decoding circuitry.

B. Two-Address Instruction Format

```
MOV R1, A       ; R1 = Mem[A]
ADD R1, B       ; R1 = R1 + Mem[B]
MOV R2, C       ; R2 = Mem[C]
ADD R2, D       ; R2 = R2 + Mem[D]
MUL R1, R2      ; R1 = R1 * R2
MOV X, R1       ; Mem[X] = R1
```

- **Pros:** Shorter instruction bit length than 3-address; highly versatile.
- **Cons:** Overwrites one of the original source operands.

C. One-Address Instruction Format (Accumulator Architecture)

```

LOAD A      ; AC = Mem[A]
ADD B       ; AC = AC + Mem[B]
STORE X     ; Mem[X] = AC
LOAD C      ; AC = Mem[C]
ADD D       ; AC = AC + Mem[D]
MUL X       ; AC = AC * Mem[X]
STORE X     ; Mem[X] = AC

```

- **Pros:** Very simple instruction word format; small CPU decoder logic.
- **Cons:** Long program length; heavy memory traffic to/from Accumulator.

D. Zero-Address Instruction Format (Stack Architecture)

```

PUSH A      ; Push A onto Stack
PUSH B      ; Push B onto Stack
ADD         ; Pop A, B; Push (A+B)
PUSH C      ; Push C onto Stack
PUSH D      ; Push D onto Stack
ADD         ; Pop C, D; Push (C+D)
MUL         ; Pop (A+B), (C+D); Push Product
POP X       ; Pop result into Mem[X]

```

- **Pros:** Compact code; ideal for expression evaluation and compilers.
- **Cons:** High stack overhead; sequential stack access can slow execution.

Q3. Explain the basic instruction cycle in detail with register transfer operations, and explain interrupt processing with a diagram.

1. Instruction Cycle Stages & Register Transfer Notation

1. Fetch Stage:

- * $MAR \leftarrow PC$
- * $MDR \leftarrow Memory[MAR]$
- * $IR \leftarrow MDR$
- * $PC \leftarrow PC + 1$

2. Decode Stage:

- * $Decoder \leftarrow IR(Opcode)$
- * $MAR \leftarrow IR(Address)$ (If memory operand)

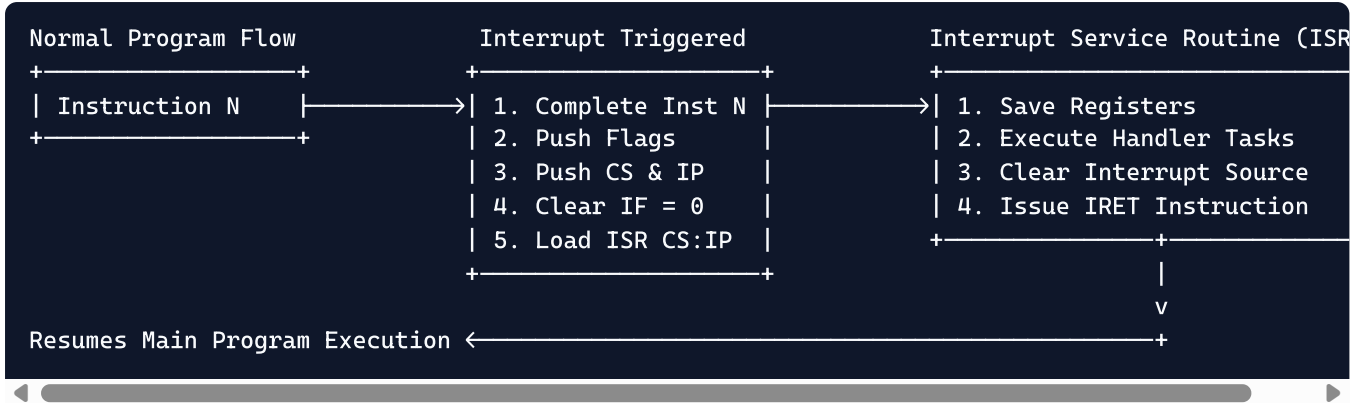
3. Execute Stage:

- * If ADD : $ALU_Out \leftarrow AC + MDR$, $AC \leftarrow ALU_Out$
- * If LOAD : $AC \leftarrow MDR$

4. Store / Write-Back Stage:

- * $Memory[MAR] \leftarrow MDR$ (If store instruction)

2. Interrupt Processing Cycle

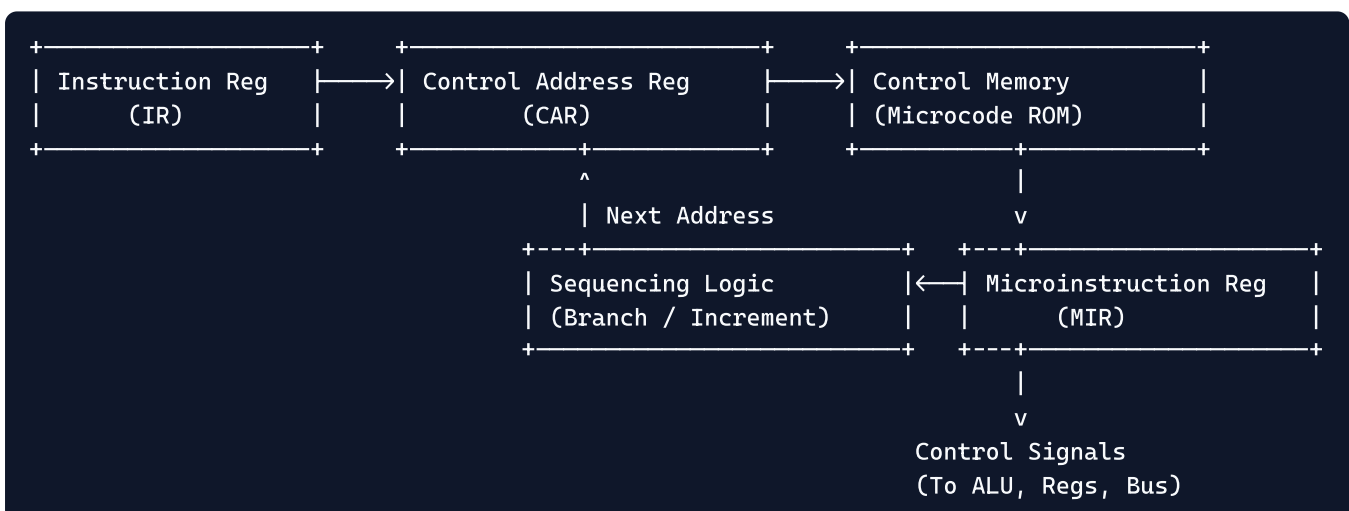


• Step-by-step:

1. CPU completes execution of current instruction N.
2. Saves return context by pushing Flag register, CS, and IP to stack.
3. Sets IF = 0 to block new lower-priority interrupts.
4. Reads Interrupt Vector from IVT to load ISR address into CS and IP.
5. Executes ISR, concluding with IRET to pop IP, CS, and Flags back, resuming normal code flow.

Q4. Explain microprogrammed control unit in detail — block diagram, working for ADD AX, BX, microinstruction format, pros/cons, and comparison with hardwired.

1. Block Diagram



2. Microinstruction Format



3. Step-by-step Execution for ADD AX, BX

- **Address 100:** Control Signals = Temp1 ← AX ; Next Address = 101.
- **Address 101:** Control Signals = Temp2 ← BX, ALU_Op = ADD ; Next Address = 102.

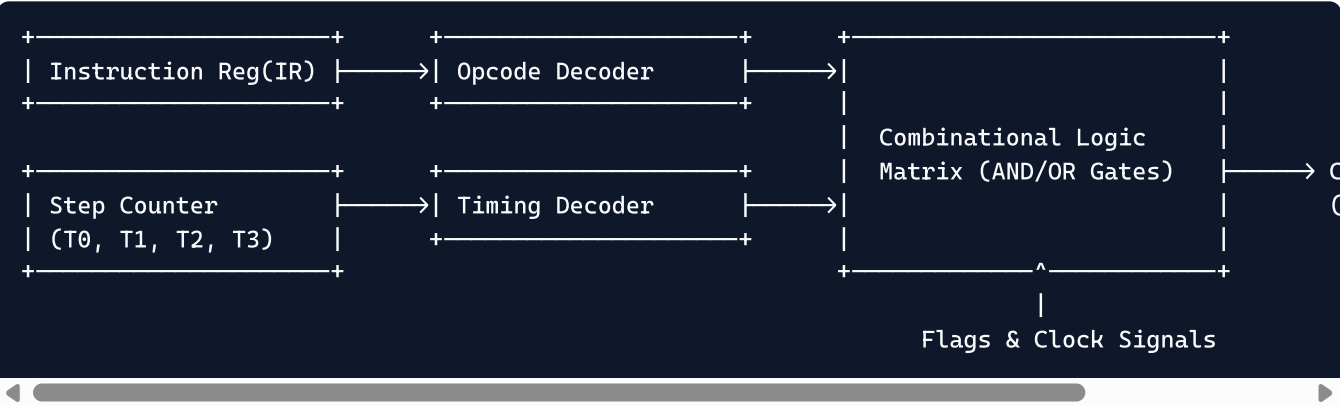
- **Address 102:** Control Signals = `AX ← ALU_Out` ; Next Address = `Map Opcode (Fetch)` .

4. Comparison Table

Feature	Hardwired Control Unit	Microprogrammed Control Unit
Implementation	Hardware combinational logic gates.	Microcode stored in Control ROM.
Speed	Maximum (direct gate propagation delay).	Slower (ROM access time delay).
Flexibility	Inflexible; requires circuit redesign.	Flexible; update microprograms in ROM.
CISC vs RISC	Ideal for RISC architecture.	Ideal for CISC architecture.

Q5. Explain hardwired control unit in detail — block diagram, working for ADD AX, BX, example of control signal generation, pros and cons.

1. Detailed Block Diagram



2. Control Signal Generation Example

Consider control signal `Ld_AX` (Load AX register):

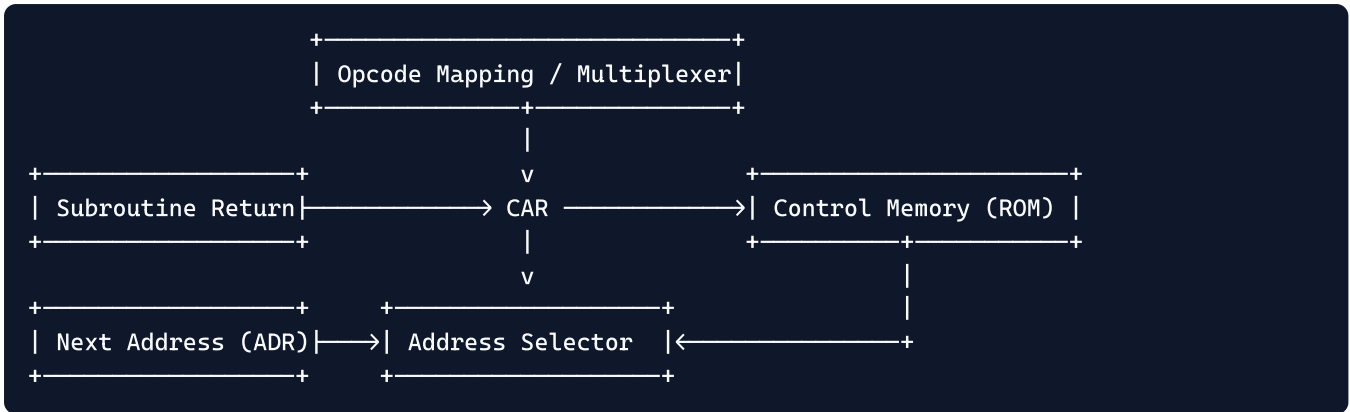
- * `Ld_AX` must trigger during `FETCH` at state `$T_1$`, or during `ADD AX, BX` at state `$T_3$`, or during `MOV AX, BX` at state `$T_2$`.
- * **Boolean Expression:**
$$\text{Ld_AX} = (\text{FETCH} \cdot T_1) + (\text{ADD} \cdot T_3) + (\text{MOV} \cdot T_2)$$
- * **Hardware Implementation:** A 3-input OR gate driven by three AND gates.

3. Pros and Cons

- **Pros:** High speed (operates at native clock rate); compact footprint for RISC.
- **Cons:** Extremely difficult to design, debug, and modify; high engineering cost.

Q6. Explain microinstruction sequencing and execution in detail with block diagram, microinstruction format, and a complete worked example for ADD AX, BX.

1. Sequencing Architecture Diagram



2. Complete Worked Example for ADD AX, BX

Assume Opcode of ADD AX, BX maps to Control Memory Address 200 :

Control Addr	Micro-Operation Signals	Condition Field (CD)	Next Addr / Sequencing
200	Reg_Read AX → Temp1	Unconditional (00)	CAR ← CAR + 1 (201)
201	Reg_Read BX → Temp2 , ALU_Op = ADD	Unconditional (00)	CAR ← CAR + 1 (202)
202	AX ← ALU_Out , Update Flags	Unconditional (00)	MAP (Jump to Fetch)

Q7. Explain parallel processing concepts in detail — sequential vs parallel, why needed, applications, bit-level and instruction-level parallelism.

1. Sequential vs. Parallel Processing

- Sequential Processing:** Executes one instruction at a time on a single CPU core. Execution time equals sum of all instruction times.
- Parallel Processing:** Concurrent execution of multiple instructions or data streams across multiple processing units to shorten overall execution time.

2. Why Parallel Processing is Needed

- Physical Power Wall:** Transistor clock rates capped at ~5 GHz due to extreme heat dissipation.
- High Throughput Demand:** Modern applications (AI, Big Data, 3D Rendering) require trillions of calculations per second.

3. Levels of Parallelism

```

BIT-LEVEL PARALLELISM:
16-bit operation: [ 16-bit Add ] + [ 16-bit Add ] (2 steps)
64-bit operation: [ 64-bit Single-Pass ALU Add ] (1 step)

INSTRUCTION-LEVEL PARALLELISM (ILP):
Non-Pipelined: [ Inst 1 ] [ Inst 2 ] [ Inst 3 ]
Pipelined:      [ Inst 1 ]
                  [ Inst 2 ]
                  [ Inst 3 ]

```

Q8. Explain Flynn's classification in detail with the classification matrix, diagrams, examples, real-life analogies, and applications for SISD, SIMD, MISD, and MIMD.

1. Flynn's 2x2 Matrix

	SINGLE DATA STREAM (SDS)	MULTIPLE DATA STREAM (MDS)
SINGLE INSTRUCTION STREAM (SIS)	SISD	SIMD
MULTIPLE INSTRUCTION STREAM (MIS)	MISD	MIMD

2. Complete Breakdown of the 4 Architectures

- SISD (Single Instruction, Single Data):**
 - * *Structure:* 1 Control Unit, 1 Processor Unit, 1 Memory Unit.
 - * *Analogy:* Single chef cooking 1 dish at a time.
 - * *Example:* Intel 8086, classic single-core CPUs.
- SIMD (Single Instruction, Multiple Data):**
 - * *Structure:* 1 Control Unit issuing 1 instruction to multiple ALUs processing different data elements.
 - * *Analogy:* Aerobics instructor shouting 1 move to 50 participants simultaneously.
 - * *Example:* Nvidia GPUs, Intel AVX-512 extensions.
- MISD (Multiple Instruction, Single Data):**
 - * *Structure:* Multiple Control/Processor Units operating different algorithms on 1 shared data stream.
 - * *Analogy:* Multiple specialists running different diagnostic tests on 1 patient's blood sample.
 - * *Example:* Spacecraft fault-tolerant flight control systems.
- MIMD (Multiple Instruction, Multiple Data):**
 - * *Structure:* Multiple independent processors executing different programs on different data.
 - * *Analogy:* Multiple chefs in a restaurant cooking different recipes for different tables.
 - * *Example:* Multi-core CPUs (Intel Core i9, AMD EPYC), Supercomputer clusters.

Q9. Explain instruction pipelining and pipeline hazards in detail — 5-stage pipeline, pipeline timing diagram, hazards, and hazard-handling techniques.

1. 5-Stage Pipeline Execution

Clock Cycle	C1	C2	C3	C4	C5	C6	C7	C8
Inst 1	IF	ID	EX	MEM	WB			
Inst 2		IF	ID	EX	MEM	WB		
Inst 3			IF	ID	EX	MEM	WB	
Inst 4				IF	ID	EX	MEM	WB

2. Pipeline Hazards & Solutions

- Structural Hazard:** Two stages need same hardware resource.
 * *Solution:* Separate L1 Code and Data caches; insertion of stall bubbles.
- Data Hazard (RAW, WAR, WAW):** Operand dependency.
 * *Solution:* Operand Forwarding (hardware bypass path from EX output to EX input), Stalling.
- Control Hazard:** Branch instructions changing PC.
 * *Solution:* Dynamic Branch Prediction, Branch Target Buffers (BTB).

Q10. Draw and explain the pipeline timing diagram for 4 instructions in a 5-stage pipeline; identify a RAW hazard scenario, show how stalling resolves it, and calculate stall cycles.

1. RAW Hazard Scenario

Consider the code:

```
I1: ADD R1, R2, R3      ; R1 is written in WB stage (C5)
I2: SUB R4, R1, R5      ; R1 is read in ID stage (C3) → RAW Hazard!
```

2. Pipeline Timing Diagram WITH STALLING (Resolving RAW Hazard)

Clock Cycle	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10
I1: ADD	IF	ID	EX	MEM	WB					
I2: SUB		IF	ID*	--	--	ID	EX	MEM	WB	
I3: OR			--	--	--	IF	ID	EX	MEM	WB

(Note: **ID*** indicates stalled decode stage. Two bubble cycles **--** are inserted).

3. Stall Cycles Calculation

- Without Forwarding: **2 Stall Cycles** are introduced to allow **I1** to write **R1** in **C5** so **I2** can safely read it in **C6**.
- Total Execution Time = $8 + 2 = 10\text{ clock cycles}$.